Task 5

Security Lab

Overview

This task focuses on implementating Zero Trust architecture which involves transit encryption using HTTP and TLS, mutual TLS handshakes between services and users, and user authentication and authorization using JWT (JSON Web Token) access tokens. A root certificate authority (CA), server certificate authority, and rest service authority are generated where server CA and rest service CA are leaf certificates which are then signed by the root CA. A JWT access token is generated with a limited time period to validate access to each of these services. Threat simulations with the wrong CA and invalid JWT are done to demonstrate failure/protection modes.

Building Certificate Authority

A root key ‘ca.key’ and certificate ‘ca.crt’ are generated using openssl alongside leaf certificate keys for server ‘grpc-service.key’ and REST service ‘rest-service.key’. The leaf certificates issue certificate signing requests ‘grpc-service.csr’ and ‘rest-service.csr’. These are then signed by the root CA to generate leaf certificates ‘grpc-service.crt’ and ‘rest-service.crt’. New directories ‘security/certs/’ are created to store these certificates and keys. The generated keys are private keys, and hence they are added to ‘.gitignore’, which prevents them from being uploaded to the remote repository to maintain privacy. ‘ca.crt’ is imported into the trust store in the user’s browser.

Command: openssl x509 -text -noout -in rest-service.crt

The above command is used to inspect the certificate and verify SAN (Subject Aternative Name) and validity dates.

Traefik Configuration

Configuration files ‘traefik.yml’ and ‘dynamic.yml’ are created in the directory ‘/security’. Traefik automatically discovers running containers on specified Docker sockets and routes all HTTP/HTTPS traffic. It is setup to listen on port 443 as it’s a universally standardized port for HTTP/HTTPS traffic. Since the user is sending requests to the REST service, the loadbalancer routes the incoming request to port 5000, where the REST service is hosted. Traefik presents certificates to browsers as ‘ca.crt’ is imported into the user’s browser. However, the REST service certificate is also accepted since it was generated using a signature from the root certificate ‘ca.crt’. Traefik ensures all the traffic from the user’s browser is encrypted in transit using certificates signed by our private CA. This is hosted on port 8080.

Keycloak Setup

Realm is an isolated security domain that acts as an identity provider. It is hosted on port 8081, which gives a login page. The developer can log in as ‘admin’ to create or remove realms. In the provided ‘real-export.json’ file, the username and password are setup as “alice@example.com” and

“secret” respectively. This file also creates the realm ‘DSA Lab Realm’. When the user wants to use the application, they login to keycloak and it generates a limited time JWT which is signed by a private cryptographic key defined in ‘real-export.json’. This JWT contains user data like usernames, roles, and other relevant data as per configuration. This JWT is appended with each request from the user, and the REST service verifies the identity using this access token to be able to perform the required operation.

Command:

```
export KC="http://localhost:8080/realms/dsa-lab/protocol/openid-connect"

curl -s -d "client_id=rest-client" -d "grant_type=password" \

  -d "username=alice@example.com" -d "password=secret" \

  "$KC/token" | jq -r .access_token > token.jwt
```

The above command is used to login to Keycloak, which gives the user the access token. This token is also pulled by the REST service for verification.

Mutual TLS: REST and gRPC Modifications

Before every request, the identity of the request sender and its authenticity need to be checked and verified. Since Keycloak issues the JWT for the relevant Realm (dsa-lab), the REST service is modified to check for the Authorization Bearer token for every incoming request, which is provided by Keycloak per user. If the token does not match, the request is rejected. Otherwise, the intended operation is performed.

Example request:

```
curl -k -H "Authorization: Bearer $(cat token.jwt)" https://localhost/api/items
```

In the above command, the generated access token is sent alongside the request to prove the user’s identity.

CA certificates and private keys are imported into both REST and gRPC services, as these credentials are used to open secure ports for both these services to be hosted. These credentials are used for a mutual handshake between internal services to be able to encrypt communication with each other.

Docker Compose Modifications

Services from Keycloak and Gateway (Traefik) are added on ports 8081 and 8080, respectively, with relevant file and directory mounting.

Control Flow

- User logs into Keycloak.
- Keycloak generates a JWT based on user data and the registered realm.
- The user uses this JWT while sending an HTTP request.

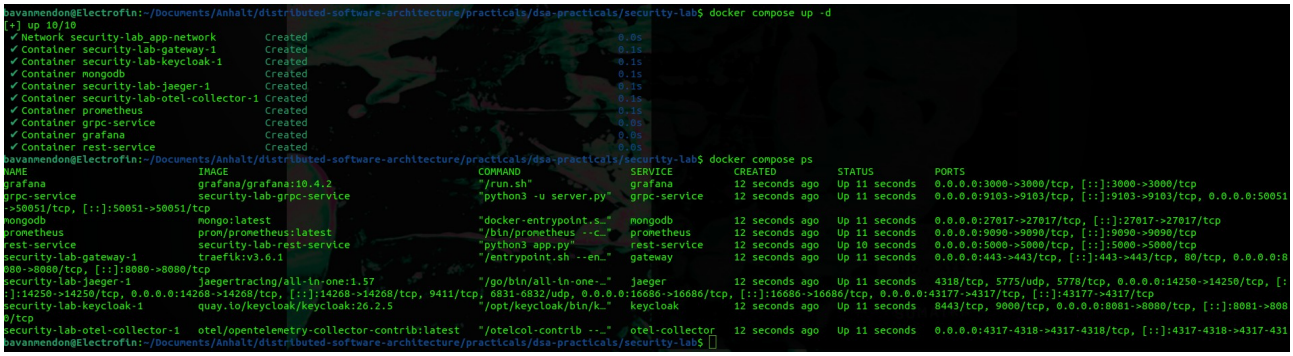
- This request is routed via Traefik from port 443 to the REST service on port 5000.
- The REST service checks for a JWT match before processing the request. If it’s a match, the operation is performed by interacting with the gRPC service
- The REST service credentials and gRPC service credentials are checked using mTLS using leaf certificates and private keys generated for both. Both the leaf certificates were created by signature from the root certificate. Hence, handshaking happens.
- The gRPC service interacts with MongoDB to retrieve requested items and sends it back to the REST service, which sends the response to the user.

Results and Failure Simulation

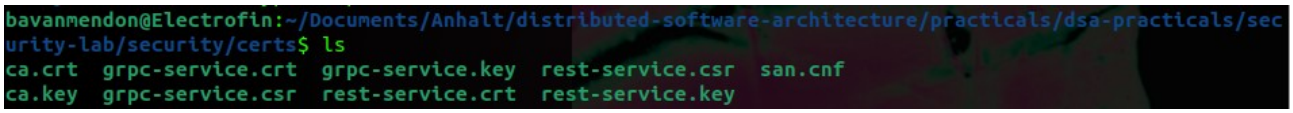
Build, start and check running containers using a compose file

The orchestrated containers are checked using the following command:

docker compose ps // List composed containers

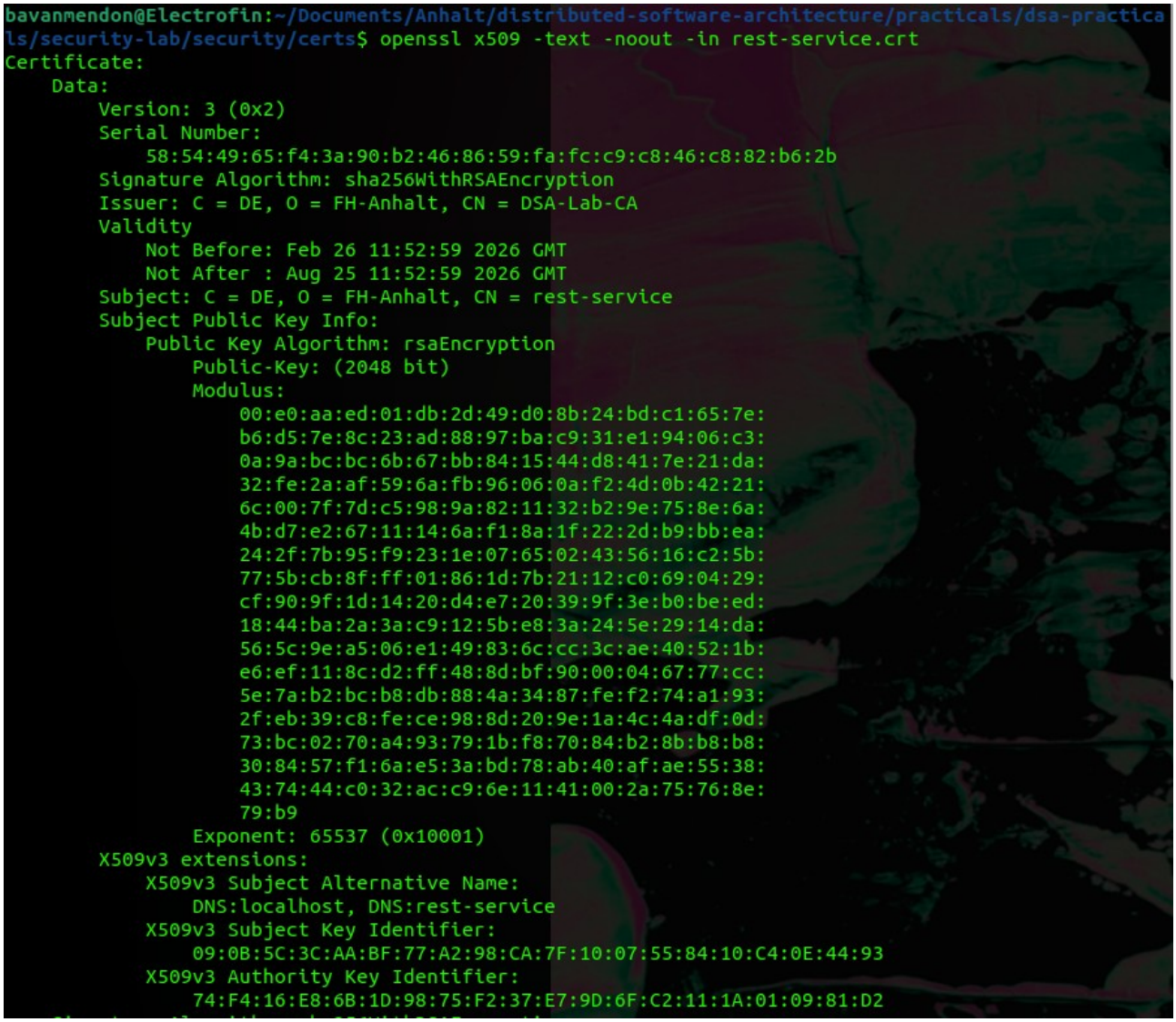


List of generated root, REST, and gRPC certificates, keys, and certificate signature requests



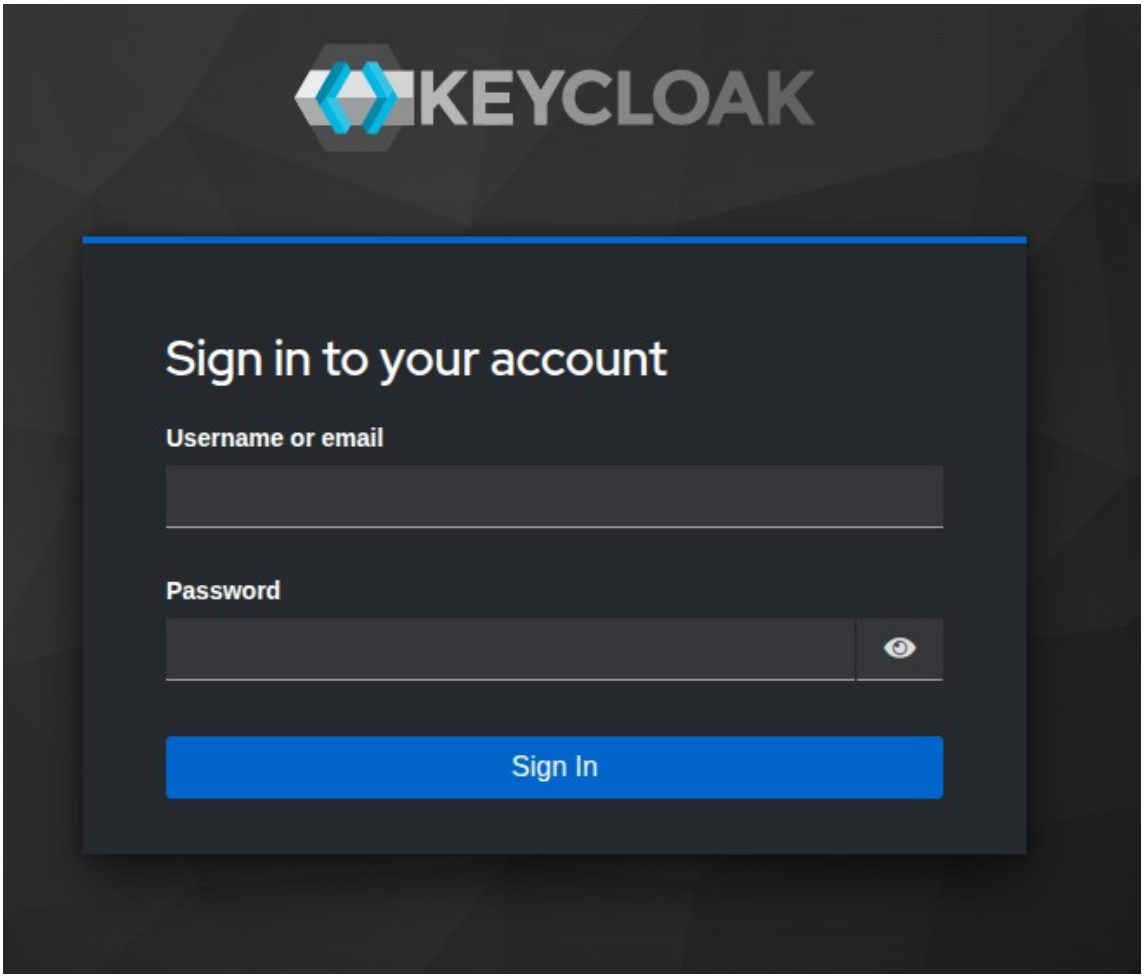
Inspection of the REST certificate by SAN and date validity verification

The inspection verified teh ‘Subject Alternate Name’ and validity dates of these certificates. The root certificate is created with a validity of 365 days, and the leaf certificates are created with a validity of 180 days.



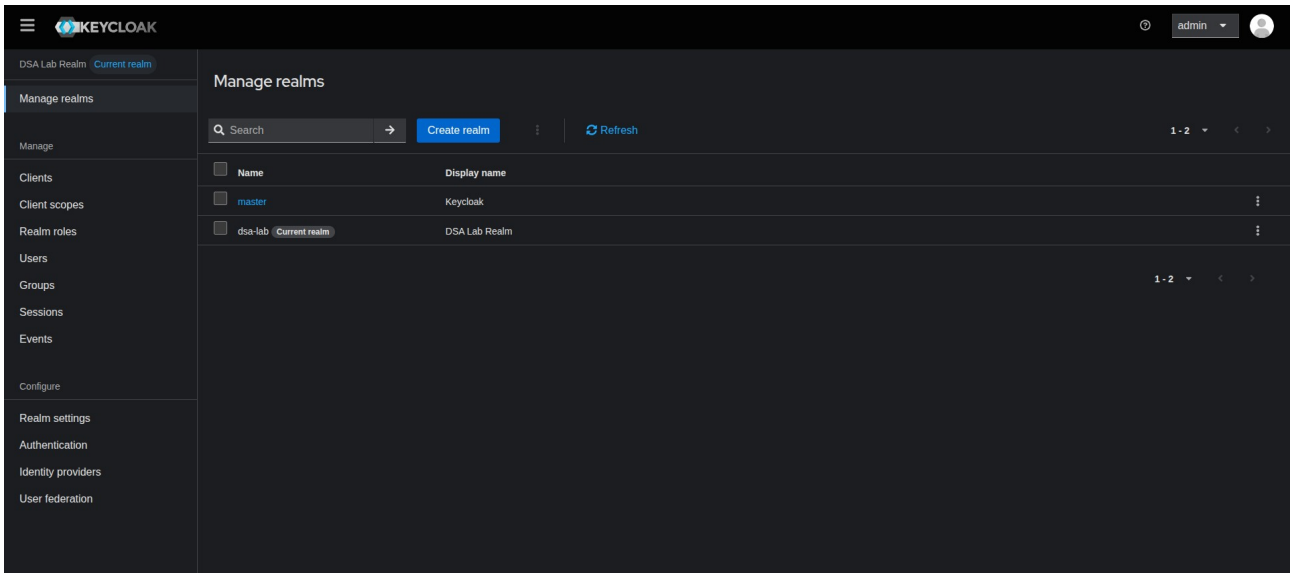
```
Signature Algorithm: sha256WithRSAEncryption
Signature Value:
    7f:0a:3d:b0:45:71:1a:74:92:72:a5:a4:79:ac:94:7e:7b:b7:
    97:bc:56:f6:63:8d:30:14:e0:f2:ea:ad:1c:dd:5b:db:3f:8d:
    f1:c3:b2:28:fd:90:49:72:5b:30:8e:27:dd:20:0e:1b:ce:5e:
    75:4a:f7:7c:25:c5:e2:05:db:aa:a0:5c:0c:0a:de:52:2b:95:
    b9:07:6d:44:d2:4d:2c:91:b5:dd:a3:84:58:24:f9:f4:59:13:
    8e:89:a5:c6:f0:b8:e2:08:ce:49:99:c1:89:d2:83:a1:5e:29:
    cc:1a:4b:d2:39:73:49:2c:04:e5:1e:a0:95:45:ac:3c:16:bf:
    31:a9:7e:01:e0:76:0d:d1:2b:03:a8:67:41:11:67:d6:c7:02:
    4b:f0:a1:e2:df:ce:79:70:ad:7d:fa:fd:61:d1:62:61:b3:05:
    44:32:37:d3:02:34:3e:cd:dc:d9:3d:70:bd:84:09:f2:de:3f:
    7f:cc:4a:c5:07:3f:56:7b:8e:a5:cd:8a:c2:33:a1:38:e7:43:
    fe:3f:cc:0a:95:95:86:61:fd:f6:75:82:47:b9:a3:50:3f:fe:
    3c:cd:c5:44:9f:80:26:6a:75:be:9f:34:ec:6a:5c:69:d9:b7:
    bf:02:69:5e:b4:2f:d0:90:48:7f:4a:27:74:0c:1d:36:d9:60:
    4f:a6:f8:8c:be:65:e6:73:ec:92:a5:d5:87:af:55:9b:c4:cc:
    50:6b:27:16:b6:5e:d8:a3:75:53:cd:9a:86:07:8b:72:9f:c1:
    a5:e9:5c:02:58:66:f2:bf:41:dd:e3:42:e6:4c:85:75:7d:39:
    10:ae:38:31:90:3c:08:8a:d5:0d:6c:3c:61:c8:d2:5a:10:49:
    1c:49:ed:65:68:d3:08:f9:72:c0:b0:b1:54:05:60:f7:1a:12:
    b6:f5:59:1c:bf:0e:06:0e:db:9d:67:54:5b:e4:4c:33:b2:4b:
    da:fc:1f:79:0a:6d:1e:27:fa:1c:bb:9c:55:26:b2:c1:50:2d:
    8f:fc:5d:dc:c5:57:d9:1b:74:31:56:40:24:07:83:b7:c0:29:
    af:d5:e6:0a:71:4a:d1:3f:41:33:f2:99:02:c4:b3:83:17:10:
    08:2f:36:b7:12:34:12:a4:14:d7:cb:aa:68:6b:1f:a0:b1:c4:
    48:0e:83:f5:d7:3a:c9:dd:f5:29:40:b0:65:44:12:a1:2f:64:
    50:77:9c:40:07:b0:a1:30:4c:fe:bf:a8:90:3a:44:78:b3:b6:
    94:60:48:b4:64:c7:1f:b9:46:f4:93:d9:fb:3c:1b:e3:3b:3d:
    58:ac:c0:12:7a:b8:e9:8d:c3:10:27:73:7a:26:f0:52:e9:d2:
```

Keycloak Login



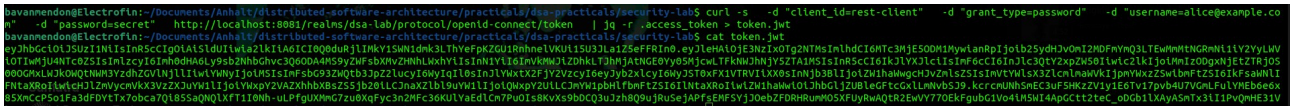
Created Realm: DSA Lab Realm

This realm is created using the file ‘real-export.json’. This file also registers access to the username “alice@example.com” with the password “secret”.



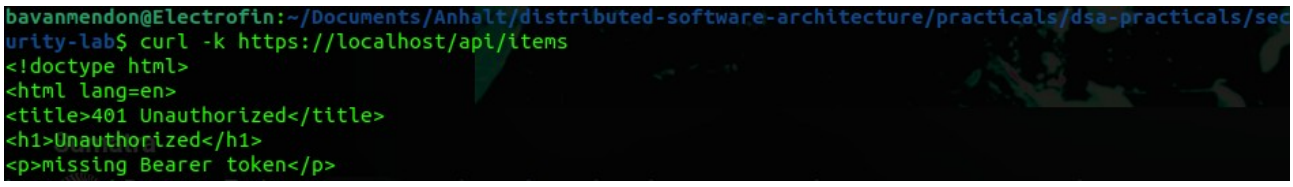
JWT Access Token generation

The JWT token is generated by logging into Keycloak using username “alice@example.com” and password “secret”, and stored in ‘token.jwt’.



User Request example without JWT

Since the request was not sent with the JWT access token, the REST service successfully prevents the requested operation.

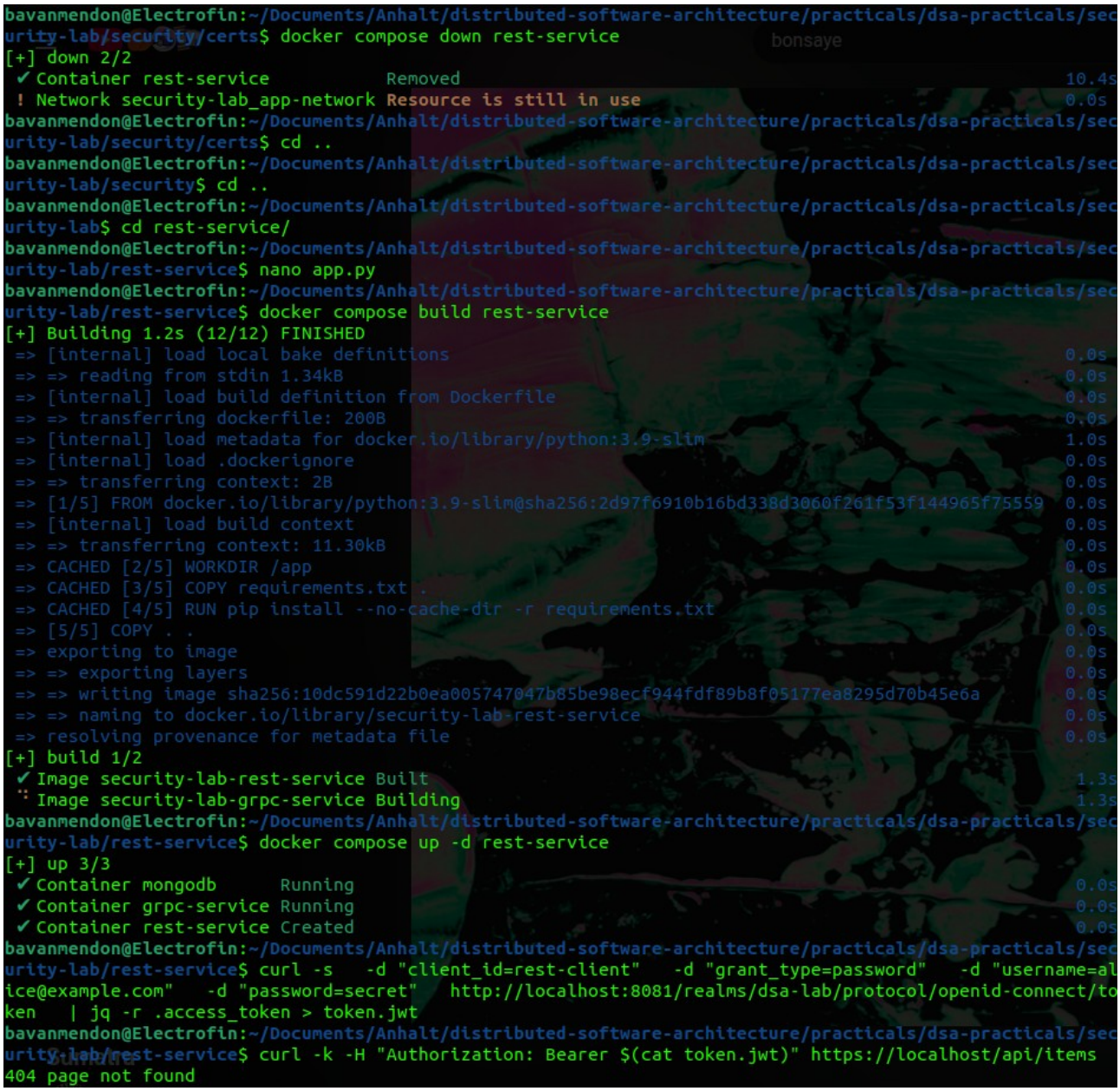


User Request example with JWT

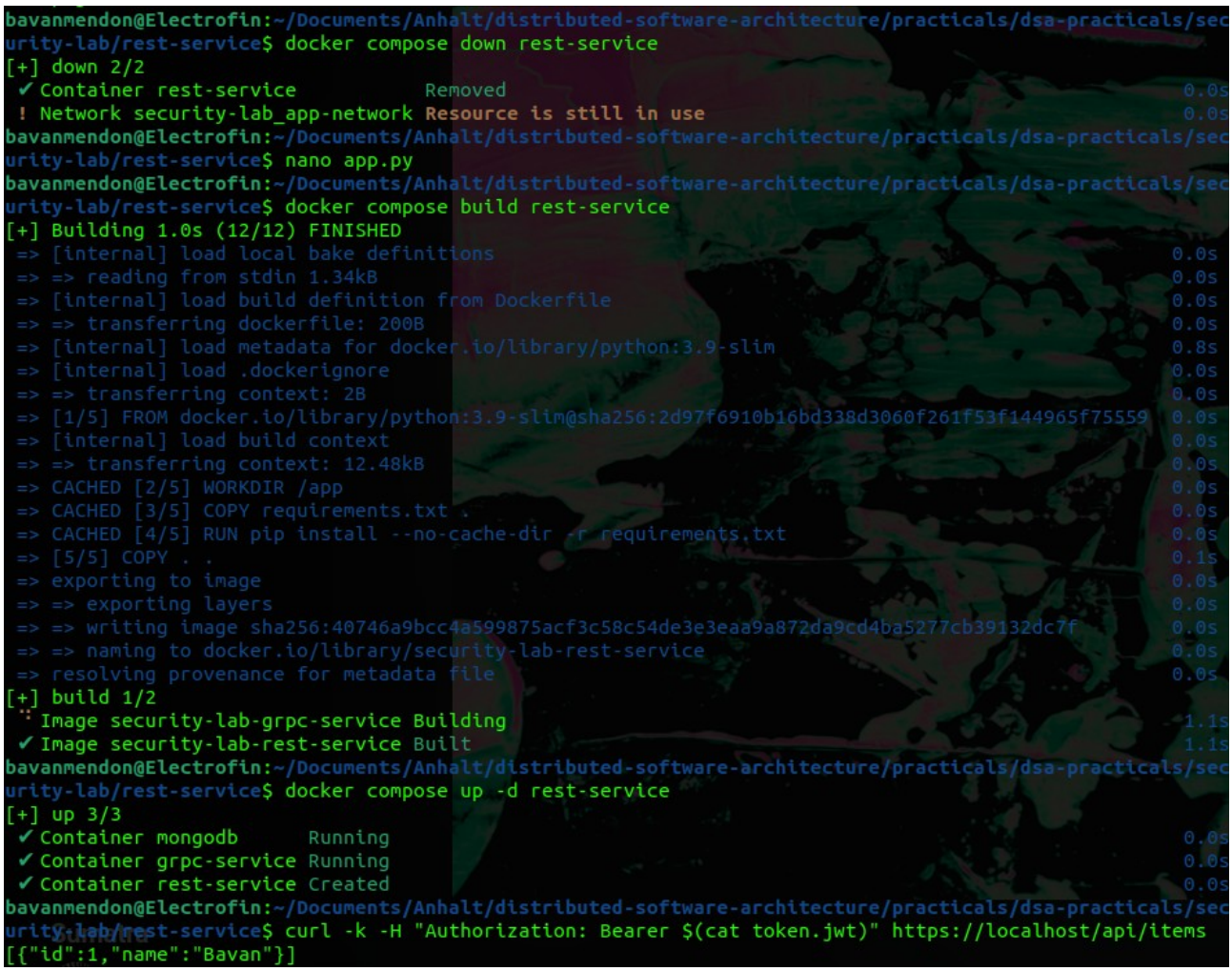
Since the request was sent with the JWT access token, the REST service performs the operation.

MTLS Test

The private key value is input as NULL in the REST service, which causes the handshake between the REST and gRPC service to fail.

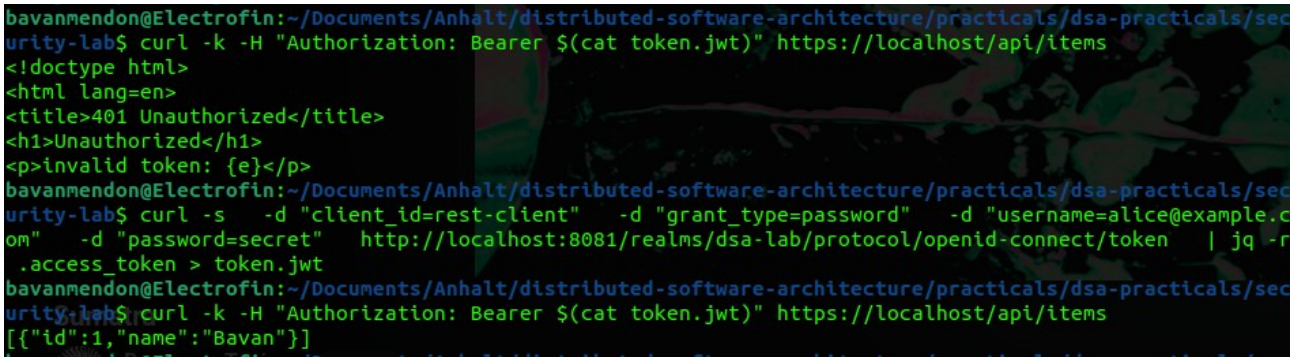


The private key is resinstated in the code again, and the handshake now works fine again.



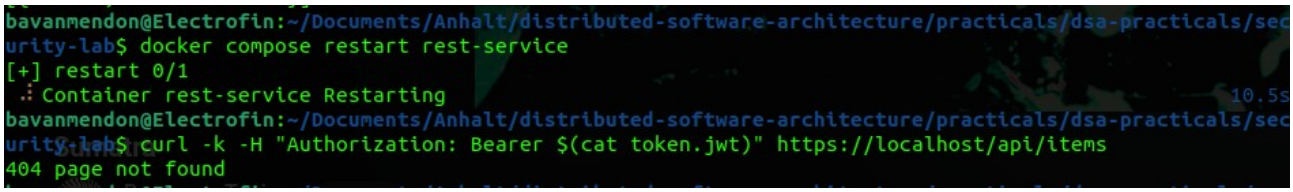
Response when JWT expires

Since the validity of the access token has expired the user request fails and a new access token is generated to perform the operation.



Response when the REST certificate name is changed

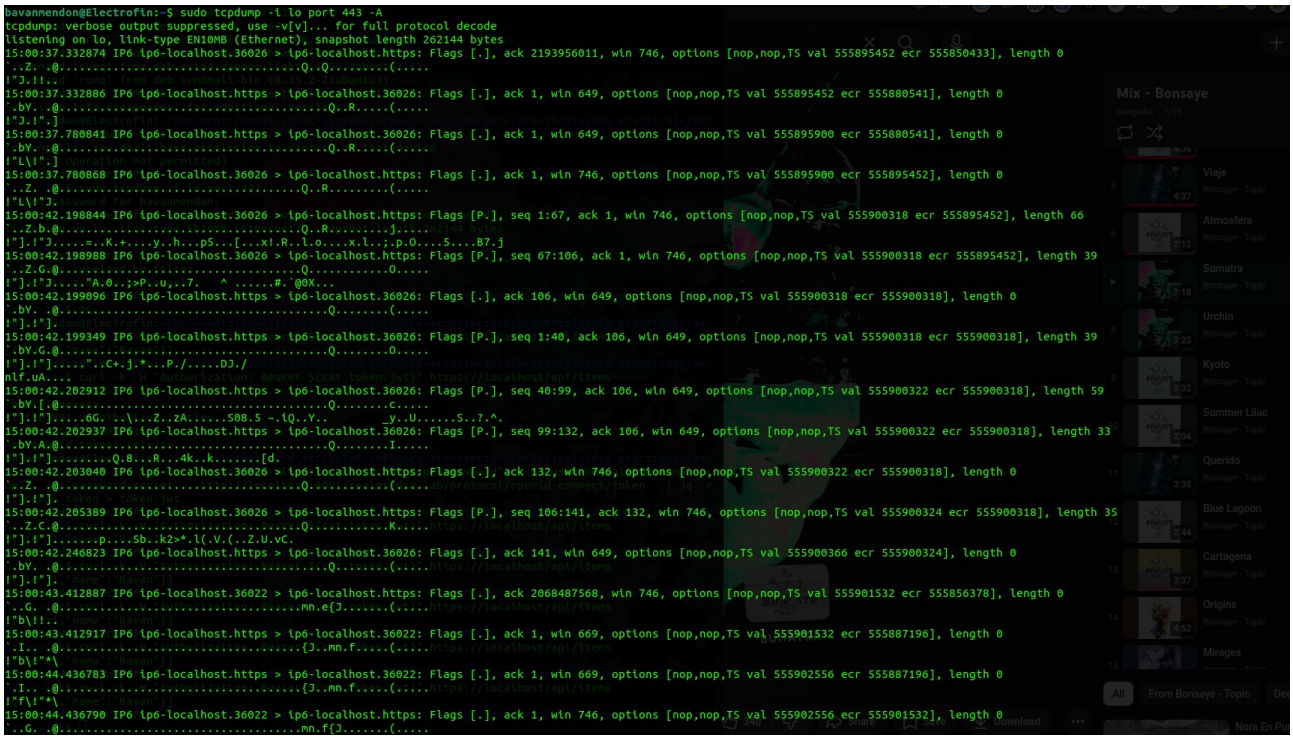
The REST service certificate name is changed, which gives an error. However, on renaming the file back to it's original name, the operation is performed succesfully.



Packet Sniffing for an encrypted payload

Command: sudo tcpdump -i lo port 443 -A // Sniff packets on port 443

Response from https://localhost/api/healthz is checked, which is supposed to be 'OK'. However, the encrypted payload is unreadable.



Browser with green lockpad

The certificate check verification shows that the connection is secure as expected after importing ‘ca.crt’ into the browser’s trust store.